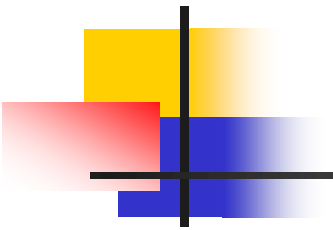




MySQL触发器

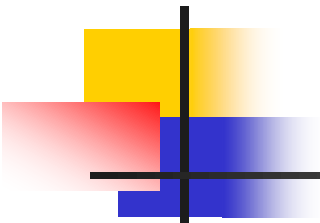


1、触发器

- **触发器(trigger)**：一种特殊的存储过程，是为保证数据完整性、确保系统正常工作而设置的一种技术。

触发器在特定的表上定义，当表上出现特定事件时将激活该触发器自动运行，以保证数据的完整性与正确性。

- **特点**
 - 触发器不需要使用CALL语句来调用
 - 触发器是事件触发的
 - 当一个预定义的事件（INSERT、UPDATE和DELETE语句）发生时，触发器会被自动调用。
 - 触发器可以查询其它表，可以包含复杂的SQL语句，主要用于满足复杂的业务规则或要求
 - 一个表可以有多个触发器，对于相同的表、相同的事件只能创建一个触发器。
 - 触发器是在数据操作有效后才执行的，即其他约束优先于触发器

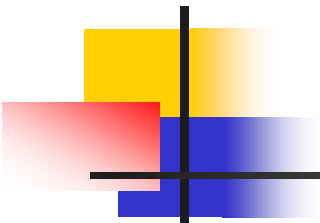


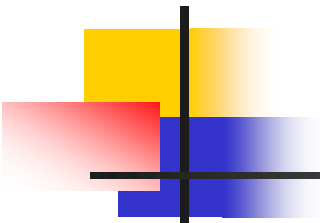
■ 触发器的主要作用

- 实现由主键和外键等约束所不能保证的复杂的参照完整性和数据一致性。
 - 触发器可以对数据库进行级联修改。
 - 实现比CHECK约束更为复杂的限制。与CHECK约束不同，在触发器中可以引用其他的表。
 - 根据表中数据修改前后的差别进行相应的操作，强制表的修改要合乎业务规则。
 - 对于一个表上的不同的操作（INSERT、UPDATE或DELETE）可采用不同的触发器，即使是对相同的语句也可调用不同的触发器完成不同的操作。

■ 触发器的典型应用

- 实现自定义完整性约束
 - 例：一位教师在同一学期最多只允许承担3门课程
- 用于值的计算
 - 例：订单明细发生改变时，重新计算订单金额并更新相关表（如订单、库存等）中的相关记录
- 日志或副本记录
 - 可确保系统跟踪和审计“时变”数据

- 
- **事件**：在MySQL事件调度器的调度下，在特定的时刻所执行的任务。因此也称为调度事件。
 - 定义事件的要素
 - 事件的“时刻”属性
 - 在某时刻仅执行一次
 - 按照时间间隔周期性地执行多次
 - 事件的“任务”属性
 - 要执行的SQL语句
 - 事件的典型应用
 - 常用于执行无人值守的系统管理任务
 - 更新汇总报告
 - 清理过期失效的数据
 - 归档、备份数据.....



2、创建触发器

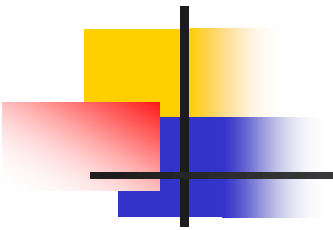
- 使用CREATE TRIGGER语句创建触发器

- 语法

CREATE

```
[DEFINER = user]
TRIGGER trigger_name
trigger_time trigger_event
ON tbl_name FOR EACH ROW
[trigger_order]
trigger_body
```

- trigger_name:要创建的触发器的名字
- trigger_time:触发器执行的时间---AFTER | BEFORE
- trigger_event:触发器触发的事件---INSERT|UPDATE|DELETE
- **FOR EACH ROW**:表示任何一条记录上的操作满足触发事件都会触发该触发器
- tbl_name:触发器关联的表
- trigger_body:触发器主体



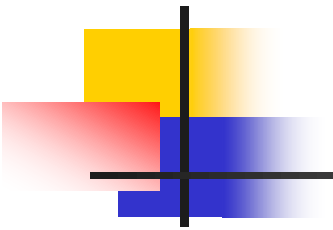
3、查看触发器

- 查看定义信息的语法

```
SHOW CREATE TRIGGER trigger_name
```

- 查看状态信息的语法

```
SHOW TRIGGERS [{FROM | IN} db_name]
```



3、删除触发器

■ 语法

```
DROP TRIGGER [IF EXISTS] [schema_name.]trigger_name
```

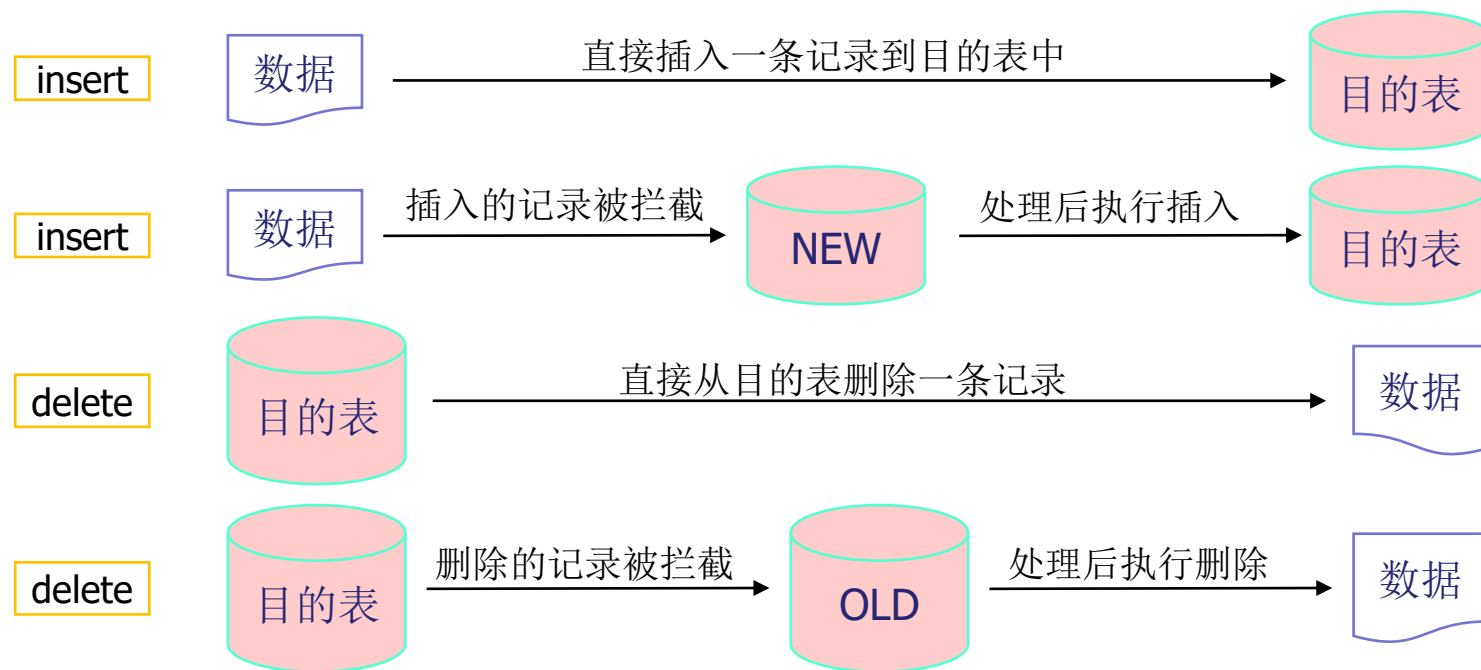
■ 注意

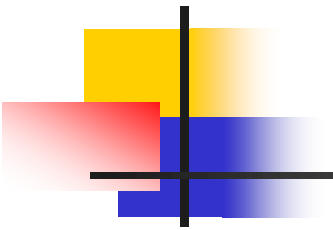
- 修改表名后，该表上的触发器仍然有效
- 删除表后，该表上创建的触发器会自动被删除

NEW、OLD

- MySQL中，触发器可以使用NEW和OLD关键字来访问受影响的新旧值。

触发器类型	NEW和OLD的使用
INSERT	NEW表示将要（BEFORE）或已经（AFTER）插入的新数据
UPDATE	OLD表示修改之前的数据，NEW表示将要或已经修改的数据
DELETE	OLD表示将要或已经删除的数据

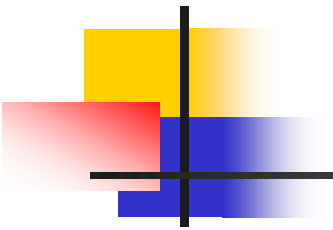




4、创建触发器实例---UPDATE触发器

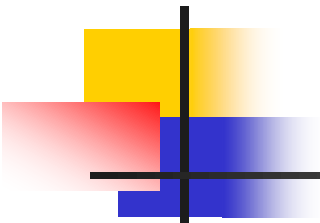
【例1】 创建一个触发器，当修改sc中成绩时，若修改后的值为无效成绩（超出[0, 100]范围）则将其“舍入”到最接近的有效成绩。

```
delimiter $  
CREATE TRIGGER valid_grade_beforeupdate_sc  
BEFORE UPDATE  
ON sc FOR EACH ROW  
BEGIN  
if NEW.grade<0 then SET NEW.grade=0;  
ELSEIF NEW.grade>100 then SET NEW.grade=100;  
END if;  
END $
```



【例2】创建一个触发器，当修改sc中成绩时，若修改后的值为无效成绩则拒绝修改操作。

```
delimiter $  
CREATE TRIGGER reject_grade_beforeupdate_sc  
BEFORE UPDATE  
ON sc FOR EACH ROW  
BEGIN  
if NEW.grade<0 or NEW.grade>100 then  
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='grade必须在【0, 100】范围内';  
END if;  
END $
```



SIGNAL 语句

- SIGNAL是MySQL中的一条SQL语句，用来中断触发器的执行。SIGNAL语句抛出一个异常，并向处理程序、应用程序的外部部分或客户提供错误信息。

- 语法

SIGNAL condition_value

[**SET** signal_information_item
[, signal_information_item] ...]

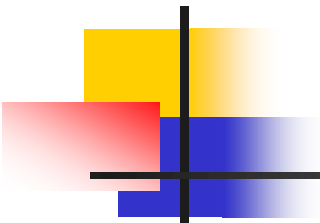
- condition_value: 表示要返回的错误值。
 - **SQLSTATE**: 信号状态码，系统变量，值是包含5个字符的字符串，代表各种错误或者警告条件的代码。成功的标志是由00000表示。‘45000’是一个通用的异常状态码，表示“未处理的用户定义异常”。
 - condition_name, 该条件名称引用了以前用 **DECLARE ... CONDITION** 定义的命名条件。
- **SET 子句**: 可设置多个条件信息项，向调用者提供提示信息。
 - **MESSAGE_TEXT**: 返回的包含错误信息的文本

4、创建触发器实例--- INSERT触发器

【例3】使用触发器保证每位学生最多选修3门课程。

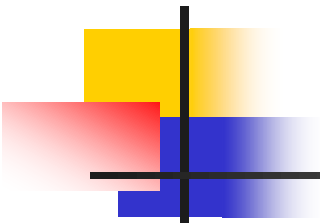
```
delimiter $
CREATE TRIGGER tr3_beforeinsert
BEFORE INSERT
ON sc FOR EACH ROW
BEGIN
if (SELECT COUNT(*) FROM sc
WHERE sno=NEW.sno)>=3 then
SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT='每位同学最多只能选修3
门课程beforeinsert';
END if;
END $
```

```
delimiter $
CREATE TRIGGER tr3_afterinsert
AFTER INSERT
ON sc FOR EACH ROW
BEGIN
if (SELECT COUNT(*) FROM sc
WHERE sno=NEW.sno)>3 then
SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT='每位同学最多只能选修
3门课程afterinsert';
DELETE FROM sc WHERE
sno=NEW.sno AND cno=NEW.cno;
END if;
END $
```



【例4】为sc创建一个触发器，当向sc中添加记录时，自动添加选课日期。

```
delimiter $  
CREATE TRIGGER T_sc_adddate  
BEFORE INSERT  
ON sc FOR EACH ROW  
BEGIN  
SET NEW.xkrq=CURDATE ( ) ;  
END $
```



4、创建触发器实例--- DELETE触发器

【例5】当删除某学生记录时，自动删除其选课记录。

```
delimiter $  
CREATE TRIGGER  
T_student_beforedel  
BEFORE DELETE  
ON student FOR EACH ROW  
BEGIN  
DELETE FROM sc WHERE  
sno=OLD.sno;  
END $
```

```
delimiter $  
CREATE TRIGGER  
T_student_afterdelete  
AFTER DELETE  
ON student FOR EACH ROW  
BEGIN  
DELETE FROM sc WHERE  
sno=OLD.sno;  
END $
```